System sterowania Ball on Beam

Raport z projektu zaliczeniowego

Piotr Bednarek, Jan Andrzejewski, Mateusz Banaszak

Styczen 2026

Spis treści

1 Wymagania projektowe **2**
1.1 Pomiar zmiennej regulowanej ze stalym okresem probkowania 2
1.2 Sterowanie w bezpiecznym zakresie zmian regulowanej zmiennej 2
1.3 Uchyb ustalony na poziomie 5% (i 1%) zakresu regulacji 3
1.4 Zadawanie wartosci referencyjnej za pomoca komunikacji szeregowej lub urzadzenia wejscia 4
1.5 Podglad sygnalow przez komunikacje szeregowa 5
1.6 Podzial kodu na moduly i dokumentacja Doxygen 5
1.7 Wykorzystanie systemu kontroli wersji 6
1.8 Dedykowana aplikacja desktopowa (GUI) 6
1.9 Skrypty do logowania sygnalow 7
1.10 Dodatkowe urzadzenie wyjscia – wyswietlacz LED 8
1.11 Dodatkowe urzadzenie wejscia – potencjometr 8
1.12 Protokol komunikacji z suma kontrolna CRC-8 9
1.13 Wykorzystanie systemu czasu rzeczywistego FreeRTOS 10

2 Podsumowanie **11**

1 Wymagania projektowe

Ponizej przedstawiono wymagania projektowe wraz z fragmentami kodu zrodlowego, ktore potwierdzaja ich spelnienie.

1.1 Pomiar zmiennej regulowanej ze stalym okresem probkowania

System dokonuje pomiaru regulowanej zmiennej (pozycji kulki) ze stalym okresem probkowania zdefiniowanym jako `PID_DT_MS = 30ms`. Zapewnia to funkcja `osDelay()` systemu FreeRTOS wywolana na koncu petli sterowania.

```c
// --- Global System Settings ---
#define PID_DT_MS           30
#define SETPOINT_DEFAULT    125.0f
#define BALL_RADIUS         20
#define AVG_ERR_SAMPLES     35
```

Listing 1: Staly okres probkowania (main.h)

```c
/* Infinite loop */
for (;;) {
    loop_counter++;

    // ... kod sterowania ...

    // Odczyt dystansu w trybie continuous
    distance = readRangeContinuousMillimeters(&distanceStr);

    // ... obliczenia PID ...

    osDelay(PID_DT_MS); // Loop delay - staly okres 30ms
}
```

Listing 2: Implementacja stalego okresu w petli sterowania (main.c)

1.2 Sterowanie w bezpiecznym zakresie zmian regulowanej zmiennej

System zapewnia bezpieczne sterowanie poprzez zdefiniowanie twardych limitow kata serwa w pliku `main.h` oraz ich egzekwowanie w kodzie sterowania.

```c
// --- Servo Configuration ---
#define SERVO_CENTER        100.0f
#define SERVO_MIN_LIMIT     60.0f
#define SERVO_MAX_LIMIT     140.0f
#define SERVO_ANGLE_DEADBAND 0.8f
```

Listing 3: Definicje limitow bezpieczenstwa serwa (main.h)

```c
// Zabezpieczenie przed NaN (Not a Number)
if (isnan(pid_angle) || isinf(pid_angle)) {
    pid_angle = SERVO_CENTER;
}

```

```c
if (pid_angle < SERVO_MIN_LIMIT)
    pid_angle = SERVO_MIN_LIMIT;
else if (pid_angle > SERVO_MAX_LIMIT)
    pid_angle = SERVO_MAX_LIMIT;

// Bezposrednie sterowanie serwem
volatile float smoothed_angle = pid_angle;
SetServoAngle(smoothed_angle);
```

Listing 4: Saturacja wyjscia PID (main.c)

1.3 Uchyb ustalony na poziomie 5% (i 1%) zakresu regulacji

System oblicza sredni blad (rolling average) z ostatnich 35 probek, co pozwala na monitorowanie uchybu ustalonego. Zakres regulacji wynosi 250mm, wiec 5% = 12.5mm, a 1% = 2.5mm. Wizualizacja na LEDach wykorzystuje te progi.

```c
// Bufor sredniego uchybu
float err_buffer[AVG_ERR_SAMPLES];
uint8_t err_idx = 0;
float err_sum = 0.0f;

// --- Obliczanie Sredniego Bledu (Rolling Average) ---
float current_abs_err = (current_error < 0) ? -current_error :
    current_error;

err_sum -= err_buffer[err_idx];       // Odejmij najstarsza probke
err_buffer[err_idx] = current_abs_err; // Zapisz nowa
err_sum += current_abs_err;           // Dodaj nowa

err_idx++;
if(err_idx >= AVG_ERR_SAMPLES) err_idx = 0;

float avg_error = err_sum / AVG_ERR_SAMPLES;
```

Listing 5: Sredni uchyb (Rolling Average) - main.c

```c
void UpdateErrorLEDs_5LED(float error) {
    // Wylacz wszystkie LEDy (PE2..PE6)
    HAL_GPIO_WritePin(GPIOE, LED_ERR_1_Pin | LED_ERR_2_Pin |
    LED_ERR_3_Pin |
                            LED_ERR_4_Pin | LED_ERR_5_Pin,
    GPIO_PIN_RESET);

    // Progi:
    // 1% z 250mm = 2.5mm
    // 5% z 250mm = 12.5mm
    float limit_green = 2.5f;
    float limit_yellow = 12.5f;

    if (error < -limit_yellow) {
        // Duzy ujemny -> Czerwona (PE2 - LED_ERR_1)
        HAL_GPIO_WritePin(GPIOE, LED_ERR_1_Pin, GPIO_PIN_SET);
    } else if (error < -limit_green) {
        // Sredni ujemny -> Zolta (PE3 - LED_ERR_2)
        HAL_GPIO_WritePin(GPIOE, LED_ERR_2_Pin, GPIO_PIN_SET);
    } else if (error <= limit_green) {
```

```
        // Maly blad (+/- 2.5mm) -> Zielona (PE4 - LED_ERR_3)
        HAL_GPIO_WritePin(GPIOE, LED_ERR_3_Pin, GPIO_PIN_SET);
    } else if (error <= limit_yellow) {
        // Sredni dodatni -> Zolta (PE5 - LED_ERR_4)
        HAL_GPIO_WritePin(GPIOE, LED_ERR_4_Pin, GPIO_PIN_SET);
    } else {
        // Duzy dodatni -> Czerwona (PE6 - LED_ERR_5)
        HAL_GPIO_WritePin(GPIOE, LED_ERR_5_Pin, GPIO_PIN_SET);
    }
}
```

Listing 6: Wizualizacja uchybu na LEDach (leds.c)

1.4 Zadawanie wartosci referencyjnej za pomoca komunikacji szeregowej lub urzadzenia wejscia

Wartosc referencyjna moze byc zadawana na dwa sposoby:

- Przez komunikacje szeregowa UART (komenda `S:wartosc`)
- Przez potencjometr podlaczony do wejscia ADC (PA0)

```
void StartDefaultTask(void const * argument) {
    for (;;) {
        // Obsluga komend UART
        if (cmd_received) {
            if (rx_buffer[0] == 'S' && rx_buffer[1] == ':') {
                g_setpoint = atof((char*) &rx_buffer[2]); // S:Setpoint
                if (g_setpoint < 0.0f)
                    g_setpoint = 0.0f;
                if (g_setpoint > 250.0f)
                    g_setpoint = 250.0f;
            }
            // ... inne komendy ...
            cmd_received = 0;
        }
        osDelay(5);
    }
}
```

Listing 7: Obsluga komendy setpointu przez UART (main.c)

```
// --- Obsluga Potencjometru ---
HAL_ADC_Start(&hadc1);
if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) {
    g_adc_raw = HAL_ADC_GetValue(&hadc1);
    float pot_setpoint = (float)g_adc_raw * 0.070818f;
    g_pot_setpoint = pot_setpoint;

    // Jesli tryb Analog, nadpisz setpoint
    if (control_mode == 1) {
        // Lekki filtr na setpoint z potencjometru
        static float pot_ema = 0.0f;
        if (pot_ema == 0.0f) pot_ema = pot_setpoint;
        pot_ema = 0.1f * pot_setpoint + 0.9f * pot_ema;
        g_setpoint = pot_ema;
```

```
    }
}
HAL_ADC_Stop(&hadc1);
```

Listing 8: Odczyt setpointu z potencjometru ADC (main.c)

1.5 Podglad sygnalow przez komunikacje szeregowa

System wysyla dane pomiarowe przez UART w formacie tekstowym z suma kontrolna CRC-8. Dane zawieraja: dystans surowy (D), setpoint (Z), kat serwa (A), dystans filtrowany (F), blad (E), sredni blad (V) oraz status sensora (S).

```
char data_buffer[64];
// D:Dist (raw); A:Angle; F:Filtered; E:Error; V:AvgError; S:Status; Z:
    Setpoint
int len = sprintf(data_buffer, "D:%d;Z:%d;A:%d;F:%d;E:%d;V:%d;S:%d",
    (int)raw_distance, (int)g_setpoint, (int)smoothed_angle,
    (int)filtered_dist, (int)current_error, (int)avg_error,
    distanceStr.rangeStatus);

uint8_t out_crc = CalculateCRC8(data_buffer, len);

sprintf(msg, "%s;C:%02X\r\n", data_buffer, out_crc);
static uint8_t u_throttle = 0;
if (++u_throttle >= 2) {
    u_throttle = 0;
    HAL_UART_Transmit(&huart3, (uint8_t*) msg, strlen(msg), 10);
}
```

Listing 9: Wysylanie danych przez UART z CRC (main.c)

1.6 Podzial kodu na moduly i dokumentacja Doxygen

Kod zrodlowy jest podzielony na osobne moduly o jasno zdefiniowanych funkcjonalnosciach. Kazdy modul posiada dokumentacje zgodna ze standardem Doxygen.

Struktura modulow STM32:

- `main.c/h` – glowna aplikacja, konfiguracja peryferiow
- `pid.c/h` – wrapper na CMSIS DSP PID
- `servo_pid.c/h` – wlasna implementacja PID z Anti-Windup
- `filters.c/h` – filtry cyfrowe (Mediana, EMA, 1-Euro)
- `crc8.c/h` – obliczanie sumy kontrolnej CRC-8
- `leds.c/h` – wizualizacja bledu na LEDach
- `calibration.c/h` – kalibracja sensora

```c
/**
    ****************************************************************************
 * @file    pid.c
 * @author  Piotr Bednarek Jan Andrzejewski Mateusz Banaszak
 * @date    Jan 8, 2026
 * @brief   Wrapper na biblioteke CMSIS DSP PID.
 *
 * Plik zawiera funkcje inicjalizujace i obslugujace regulator PID
 * z wykorzystaniem zoptymalizowanej biblioteki matematycznej ARM (CMSIS
    DSP).
    ****************************************************************************
 */

/**
 * @brief Inicjalizuje regulator PID.
 *        Ustawia wspolczynniki wzmocnienia oraz limity wyjscia (
    nasycenie).
 *        Wywoluje arm_pid_init_f32 aby zresetowac stan wewnetrzny
    algorytmu.
 * @param pid Wskaznik do struktury PID_Controller_t.
 * @param Kp Wzmocnienie proporcjonalne.
 * @param Ki Wzmocnienie calkujace.
 * @param Kd Wzmocnienie rozniczkujace.
 * @param min_out Dolny limit wyjscia.
 * @param max_out Gorny limit wyjscia.
 */
void PID_Init(PID_Controller_t *pid, float Kp, float Ki, float Kd,
              float min_out, float max_out) {
    // ...
}
```

Listing 10: Przyklad dokumentacji Doxygen (pid.c)

1.7 Wykorzystanie systemu kontroli wersji

Projekt wykorzystuje system kontroli wersji Git. Repozytorium znajduje sie w katalogu glownym projektu (`.git/`).

1.8 Dedykowana aplikacja desktopowa (GUI)

Aplikacja desktopowa zostala napisana w jezyku Python z wykorzystaniem frameworka PySide6 (Qt). Umozliwia ona:

- Polaczenie z mikrokontrollerem przez port szeregowy
- Wizualizacje sygnalow w czasie rzeczywistym (wykresy PyQtGraph)
- Zadawanie wartosci referencyjnej i parametrow PID
- Nagrywanie i eksport danych do CSV

- Sterowanie trybem pracy regulatora (Start/Stop)

```python
from PySide6.QtWidgets import QMainWindow, QWidget, QVBoxLayout,
    QHBoxLayout
from serial_manager import SerialManager
from widgets.metrics_panel import MetricsPanel
from widgets.control_panel import ControlPanel
from widgets.charts_panel import ChartsPanel
from widgets.terminal import Terminal

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Panel Sterowania Ball on Beam")
        self.resize(1280, 800)

        # --- Managers ---
        self.serial = SerialManager()
        self.data_history = []

        # --- Recording State ---
        self.is_recording = False
        self.recording_data = []

        # ... konfiguracja UI ...
```

Listing 11: Glowna klasa aplikacji GUI (app.py)

```python
class SerialManager(QObject):
    """
    High-level manager for Serial communication.
    Handles Thread management, Parsing, and Signal emission.
    """
    # Signals for UI
    connected = Signal(bool)
    rx_log = Signal(str, str)  # msg, type (info/error/success)
    tx_log = Signal(str)
    new_data = Signal(dict)    # Parsed data object
    ports_listed = Signal(list)

    def send_setpoint(self, val):
        self.send_command(f"S:{val:.1f}")

    def send_pid(self, kp, ki, kd):
        self.send_pid_p(kp)
        QThread.msleep(500)
        self.send_pid_i(ki)
        QThread.msleep(500)
        self.send_pid_d(kd)
```

Listing 12: Manager komunikacji szeregowej (serial_manager.py)

1.9 Skrypty do logowania sygnalow

Aplikacja desktopowa posiada wbudowany mechanizm nagrywania danych pomiarowych do plikow CSV z mozliwoscia eksportu.

```python
def _save_recording(self):
    # Generate default filename with timestamp
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    default_filename = f"nagranie_{timestamp}.csv"

    # Open file dialog
    filepath, _ = QFileDialog.getSaveFileName(
        self,
        "Zapisz nagranie",
        default_filename,
        "CSV Files (*.csv);;All Files (*)"
    )

    if filepath:
        try:
            with open(filepath, 'w', newline='', encoding='utf-8') as f:
                fieldnames = ['time', 'distance', 'filtered',
                              'setpoint', 'error', 'control']
                writer = csv.DictWriter(f, fieldnames=fieldnames)
                writer.writeheader()
                writer.writerows(self.recording_data)
        except Exception as e:
            self.terminal.append_rx(f"Blad zapisu: {e}", "error")
```

Listing 13: Funkcja zapisu nagrania do CSV (app.py)

1.10 Dodatkowe urzadzenie wyjscia – wyswietlacz LED

System posiada 5 diod LED (PE2–PE6) wizualizujacych aktualny uchyb regulacji:

- LED zielona (PE4) – blad w zakresie ±1% (2.5mm)
- LEDy zolte (PE3, PE5) – blad w zakresie ±5% (12.5mm)
- LEDy czerwone (PE2, PE6) – blad wiekszy niz 5%

```c
// --- LED Error Bar ---
#define LED_ERR_PORT      GPIOE
#define LED_ERR_1_Pin     GPIO_PIN_2
#define LED_ERR_2_Pin     GPIO_PIN_3
#define LED_ERR_3_Pin     GPIO_PIN_4
#define LED_ERR_4_Pin     GPIO_PIN_5
#define LED_ERR_5_Pin     GPIO_PIN_6
```

Listing 14: Definicje pinow LED (main.h)

1.11 Dodatkowe urzadzenie wejscia – potencjometr

System wykorzystuje potencjometr podlaczony do wejscia analogowego PA0 (ADC1_CH0) do zadawania wartosci referencyjnej w trybie analogowym.

```c
static void MX_ADC1_Init(void) {
    ADC_ChannelConfTypeDef sConfig = {0};

    hadc1.Instance = ADC1;
```

```
    hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
    hadc1.Init.Resolution = ADC_RESOLUTION_12B;
    hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
    hadc1.Init.ContinuousConvMode = DISABLE;
    // ...

    sConfig.Channel = ADC_CHANNEL_0;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_480CYCLES;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
}
```

Listing 15: Konfiguracja ADC dla potencjometru (main.c)

1.12 Protokol komunikacji z suma kontrolna CRC-8

Komunikacja szeregowa wykorzystuje sume kontrolna CRC-8 z wielomianem 0x07 dla zapewnienia integralnosci danych. Implementacja jest identyczna po stronie STM32 (C) i aplikacji desktopowej (Python).

```
/**
 * @brief Oblicza CRC-8 dla ciagu danych.
 *        Algorytm XOR z przesunieciami bitowymi.
 */
uint8_t CalculateCRC8(const char *data, int len) {
    uint8_t crc = 0x00;
    for (int i = 0; i < len; i++) {
        crc ^= data[i];
        for (uint8_t j = 0; j < 8; j++) {
            if (crc & 0x80)
                crc = (crc << 1) ^ 0x07;
            else
                crc <<= 1;
        }
    }
    return crc;
}
```

Listing 16: Implementacja CRC-8 (crc8.c)

```
def calculate_crc8(text: str) -> int:
    """
    Calculates 8-bit CRC for the given string using polynomial 0x07.
    Consistent with the STM32 implementation.
    """
    crc = 0x00
    if not text:
        return 0

    for char in text:
        byte = ord(char)
        crc ^= byte
        for _ in range(8):
            if crc & 0x80:
                crc = (crc << 1) ^ 0x07
            else:
                crc <<= 1
```

```
            crc &= 0xFF

    return crc
```

Listing 17: Implementacja CRC-8 w Pythonie (utils/crc8.py)

1.13 Wykorzystanie systemu czasu rzeczywistego FreeRTOS

System wykorzystuje FreeRTOS jako system operacyjny czasu rzeczywistego. Zdefiniowane sa dwa taski:

- `ControlTask` (priorytet wysoki) – petla sterowania PID
- `DefaultTask` (priorytet normalny) – obsluga komend UART i LEDow

```
/* Create the thread(s) */
/* definition and creation of defaultTask */
osThreadDef(defaultTask, StartDefaultTask, osPriorityNormal, 0, 256);
defaultTaskHandle = osThreadCreate(osThread(defaultTask), NULL);

/* definition and creation of ControlTask */
osThreadDef(ControlTask, StartControlTask, osPriorityHigh, 0, 1024);
ControlTaskHandle = osThreadCreate(osThread(ControlTask), NULL);

/* Start scheduler */
osKernelStart();
```

Listing 18: Tworzenie taskow FreeRTOS (main.c)

```
/**
 * @brief Function implementing the ControlTask thread.
 * @param argument: Not used
 * @retval None
 */
void StartControlTask(void const * argument) {
    // Uruchomienie PWM dla serwa
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);

    // Inicjalizacja czujnika VL53L0X
    initVL53L0X(1, &hi2c1);
    startContinuous(0);

    // Inicjalizacja kontrolera PID
    PID_Init(&g_pid_ctrl, g_Kp, g_Ki, g_Kd, SERVO_MIN_LIMIT,
   SERVO_MAX_LIMIT);
    ServoPID_Init(&g_servo_pid);

    for (;;) {
        // Petla sterowania PID
        distance = readRangeContinuousMillimeters(&distanceStr);

        // ... filtrowanie i kalibracja ...

        // Obliczenie PID
        pid_output = ServoPID_Compute(&g_servo_pid, error, filtered_dist
   );
        SetServoAngle(pid_output);
```

```

        osDelay(PID_DT_MS);
    }
}
```

Listing 19: Task sterowania PID (main.c)

```
/* Includes */
#include "FreeRTOS.h"
#include "task.h"
#include "main.h"

/* GetIdleTaskMemory prototype (linked to static allocation support) */
void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer,
                                   StackType_t **ppxIdleTaskStackBuffer,
                                   uint32_t *pulIdleTaskStackSize);

static StaticTask_t xIdleTaskTCBBuffer;
static StackType_t xIdleStack[configMINIMAL_STACK_SIZE];

void vApplicationGetIdleTaskMemory(StaticTask_t **ppxIdleTaskTCBBuffer,
                                   StackType_t **ppxIdleTaskStackBuffer,
                                   uint32_t *pulIdleTaskStackSize) {
    *ppxIdleTaskTCBBuffer = &xIdleTaskTCBBuffer;
    *ppxIdleTaskStackBuffer = &xIdleStack[0];
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
```

Listing 20: Konfiguracja FreeRTOS (freertos.c)

2 Podsumowanie

Projekt spelnia wszystkie wymagania funkcjonalne i dodatkowe:

1. **Pomiar ze stalym okresem** – `osDelay(PID_DT_MS)` zapewnia 30ms okres probkowania
2. **Bezpieczny zakres sterowania** – limity serwa 60–140 stopni
3. **Uchyb ustalony** – rolling average z 35 probek, wizualizacja na 5 LEDach (1%/5% progi)
4. **Zadawanie wartosci referencyjnej** – UART (komenda S:) i potencjometr (ADC PA0)
5. **Podglad sygnalow** – ramki UART z danymi D/Z/A/F/E/V/S + CRC-8
6. **Modularnosc i Doxygen** – 7 modulow C, pelna dokumentacja API
7. **Kontrola wersji** – repozytorium Git
8. **Aplikacja desktopowa** – PySide6 (Qt) z wykresami PyQtGraph
9. **Logowanie sygnalow** – eksport do CSV z timestampami

10. **Urzadzenie wyjscia** – 5 diod LED (PE2–PE6)
11. **Urzadzenie wejscia** – potencjometr na ADC1_CH0
12. **Suma kontrolna CRC-8** – wielomian 0x07, zgodna implementacja STM32/Python
13. **FreeRTOS** – 2 taski (ControlTask, DefaultTask)